GPS-Denied Autonomous Navigation

Map-relative localization with stereo VIO and a rangefinder

Team Talos

DroneHacks 2026

Sky Beta · PX4 · cuVSLAM · ROS 2

The problem

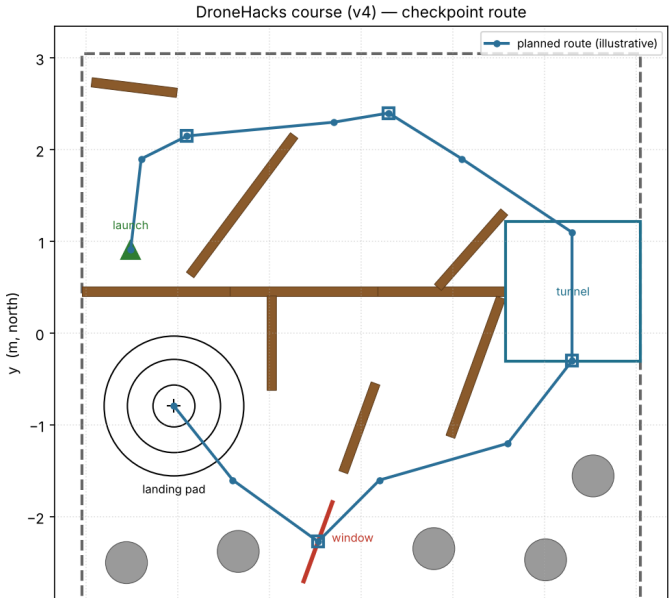
Fly a known indoor course with no GPS — pass each checkpoint in order, land inside the pad.

Three facts shaped every design choice:

- **No GPS, indoors** — position comes entirely from onboard sensing.
- **The course is known** — walls, tunnel, window, boulders and pad are surveyed into a map.
- **We do not tune the inner loop** — our scope is localization, mapping and mission; PX4 owns attitude.

Navigation is a **localize-into-a-known-map** problem — not a learn-the-world problem.

The course



The mission crosses each checkpoint in order:

1. launch
2. first corridor
3. gate A
4. tunnel
5. window
6. land in pad

Walls, tunnel, window, boulders and pad are surveyed into our map; the route is planned

Sensors — we used the kit, and used it well

No added hardware. Two sensors, each doing the one job it is good at:

Stereo camera

- drives **cuVSLAM** visual odometry (relative pose)
- depth → wall points for **map matching** (absolute correction)

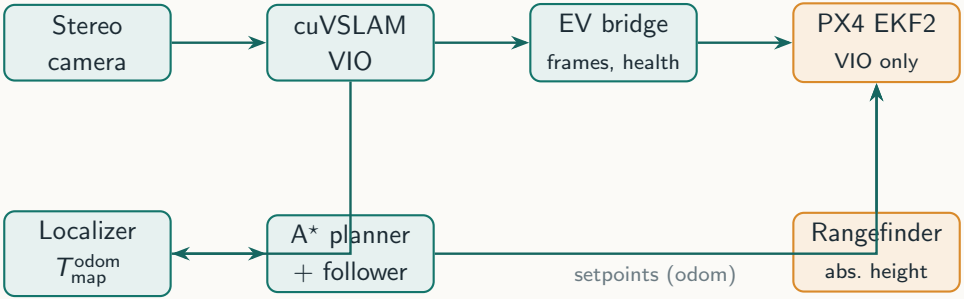
Downward rangefinder

- **absolute height** above the floor
- anchors the climb target and the landing descent, independent of VIO vertical drift

Why this matters (rule 4.6)

Each sensor covers the other's weakness: stereo gives *smooth relative* motion but drifts; the rangefinder gives an *absolute vertical* reference the camera cannot. We engineered the **use**, not the bill of materials.

System architecture



Key idea: PX4 fuses *only* raw VIO. The map never touches PX4 — corrections move the **targets** we send, through $T_{\text{map}}^{\text{odom}}$.

Localization: VIO for motion, the map for truth

Two frames, one transform

- cuVSLAM gives a *smooth but drifting* pose in its own start frame.
- We match depth points to the *known wall faces* (point-to-line ICP) to estimate $T_{\text{map}}^{\text{odom}}$.
- Setpoints are chosen in map frame, then pushed to odom for PX4.

Guarded corrections

- reject a fit if median residual > 6 cm
- Mahalanobis step gate (4σ)
- **hard cap:** no single fit moves the map > 0.30 m / 8°

These exist because a near-ground fit once mistook the *floor* for a wall and shifted the map 0.40 m.

Engineering for robustness — a failure we fixed

What happened (lab run). Near the ground the floor returned as "wall" points. The localizer accepted fits that moved the map **0.40 m / 5.8° on a stationary drone** — and the mission steered into the first-corridor wall.

What we changed:

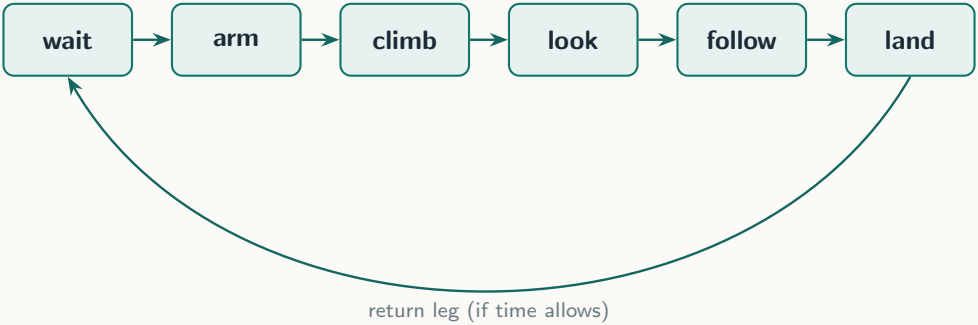
- **Floor cut** from the takeoff height — floor points stop masquerading as walls.
- **Residual + step gates + hard cap** — a bad fit can no longer yank the map.
- **Skip frames while yawing** $> 15°/\mathbf{s}$ — fast rotation stops corrupting VIO.
- **Stuck-from-setpoint watchdog** — the vehicle reacts instead of drifting silently.

The estimator's job is as much **rejecting** bad data as using good data.

Key design decisions — and paths we rejected

Option	Call	Why
ArUco / fiducials	**Rejected**	No markers on course; forward camera can't see a floor mark; single-marker yaw is unreliable; the map already gives fixed points.
Tilt to "see" the pad	**Rejected**	Pitch isn't an independent command for us; geometrically insufficient; aggressive attitude risks VIO loss.
Camera-scan tunnel	**Changed**	Parallel walls + sparse features stressed the estimator; switched to stop-and-re-align on surveyed tunnel ends.
Ground-effect handling	**Not needed**	Already covered by PX4's takeoff behavior (control we don't own).
Land inside the pad	**Retained**	Pad is a surveyed coordinate; land commit gated on a tight estimate; rangefinder sets the descent.

Mission flow



Each phase advances on an explicit condition, with timeouts and a **hover guard** as the last safety layer. The rangefinder anchors **climb** (reach true height) and **land** (descend to the floor).

launch → first corridor → gate A → tunnel → window → pad

Results

- Full out-and-return mission in simulation: **[arm-to-disarm time]**
- Checkpoints passed in order: **[N / N]**
- Map correction tightened position to **[sigma]** at the gates
- Landing: **[inside pad — which ring?]**

Present only what is honestly demonstrated — judges ask follow-ups. If a full run isn't complete, state the real status ("full mission in simulation; hardware integration in progress") rather than a number you can't defend.

What we'd do next

- **Airframe-correct planning margins** — size obstacle inflation to Sky Beta (smaller than the default), tightening the first-corridor path.
- **Richer anchoring at feature-poor stretches** — use the tunnel's and window's known geometry to pin the along-corridor direction the walls leave weak.
- **Log-driven tuning** — overlay estimate vs. map vs. plan per run to target the segments where uncertainty grows.

Thank you

Questions?

Team Talos · GPS-Denied Autonomous Navigation

Backup: why PX4 sees only VIO

Feeding a *corrected* pose to PX4 would inject map jumps into the estimator PX4 uses for control — risky, and it fights the autopilot's own reset logic. Instead we keep PX4 on smooth raw VIO and move only the **targets** we send, via $T_{\text{map}}^{\text{odom}}$. When PX4 legitimately resets its local frame, we move the transform with it (not a correction), so the map stays put.

Backup: the correction gates, in numbers

Median residual reject	> 6 cm
Mahalanobis step gate	4σ
Hard per-fit cap	0.30 m / 8°
Covariance floor	2 cm / 0.5°
Skip frames while yawing	$> 15^\circ/\text{s}$
Stuck-from-setpoint watchdog	> 0.4 m vert. / 1.0 m horiz. for 2 s